

RAG技术必背内容

1. RAG核心概念

- **定义：**RAG（检索增强生成）是一种让大模型“先查资料再回答”的技术架构，通过检索外部知识库获取相关文档，再基于检索结果生成回答，有效解决大模型的幻觉和知识滞后问题。
- **核心流程：**用户提问 → 检索知识库 → 获取相关文档 → 喂给LLM → 生成回答。
- **四大优势：**实时更新知识、减少幻觉、低成本维护、保障隐私。

2. LangChain核心对象

- **Document对象：**LangChain中文本的统一接口，包含`page_content`（内容）和`metadata`（元数据如来源、页码）。
- **用途：**所有加载器输出均为Document列表，后续分块、向量化均基于此对象。

3. 文档加载（Document Loading）

- **多格式加载器：**
 - PDF: `PyPDFLoader`
 - 网页: `WebBaseLoader`（可用CSS选择器精准提取）
 - Markdown/代码: 自定义解析器按标题或文件类型分块
 - CSV/JSON: `CSVLoader`、自定义`SimpleJSONLoader`
- **进阶技巧：**批量加载用异步（`aiohttp`）、错误处理用`tenacity`重试、缓存加载结果避免重复解析。

4. 文本分块（Text Splitting）

- **关键参数：**
 - `chunk_size`: 推荐256-512 Token，平衡语义完整与检索精度
 - `chunk_overlap`: 10-20%重叠，防止语义断裂
 - `separators`: 按`\n\n` > `\n` > `。>？` 优先级递归分割
- **进阶策略：**
 - 语义分块: 按主题边界分割（论文、长报告）
 - 基于标记分块: `MarkdownHeaderTextSplitter`按标题层级分割（技术文档）
 - 固定分块: `CharacterTextSplitter`（日志、代码）

5. 向量化与向量存储

- **Embedding模型选型：**

- 云端API: `OpenAIEmbeddings` (开箱即用, 按量付费)
- 本地部署: `OllamaEmbeddings` (`bge-m3`) 或 `HuggingFaceEmbeddings` (`bge-large-zh-v1.5`)
- 轻量模型: `MiniLM-L6` (移动端/边缘设备)

- **向量数据库选型：**

- Chroma: 轻量易用, 原型验证
- FAISS: 高性能内存检索, 离线批量处理
- Milvus: 生产级部署, 支持混合查询

- **调优要点：**

- 索引参数 (`HNSW`的`ef`、`IVF`的`nlist`) 影响精度/速度
- 检索参数 (`k`值、`nprobe`) 平衡召回率与延迟
- 混合检索: 向量+关键词融合, 互补短板

6. 传统RAG检索链

- **四步构建：**

- 加载向量库 → 2. 创建检索器 (`as_retriever`) → 3. 定义Prompt (防幻觉模板) → 4. 组装RAG链 (`create_retrieval_chain`)

- **chain_type模式：**

- `stuff`: 所有文档塞入一次prompt (少文档推荐)
- `map_reduce`: 逐块回答→合并摘要 (多文档综合)
- `refine`: 逐块迭代优化答案
- `map_rerank`: 逐块打分→选最优

7. Agentic RAG核心思想

- **从“条件反射”到“自主判断”：**Agent先推理规划 (何时检索、检索什么), 再执行, 具备反思与迭代能力。
- **三大构建模式：**
 - 单任务决策Agent: 判断是否需要检索

- 多步任务编排Agent：复杂流程用StateGraph (LangGraph)
- ReAct Agent：推理+行动循环，适合开放式分析
- **工具定义原则：**
 - 输入用Pydantic严格约束 (args_schema)
 - 描述精确，避免滥用（如“仅在确信知识库有信息时调用”）
 - 安全工具：计算器禁用危险操作 (ast安全解析)

8. 高级RAG技术

- **混合检索：**向量检索（语义广度）+ BM25（精确召回），加权逆序融合（Weighted RRF）。
- **重排序：**用Cross-Encoder深度交互式打分，精准识别内容支持度，设score_threshold过滤低相关文档。
- **查询重写：**消解多轮对话中的代词与省略，恢复完整可检索问题。
- **结构化输出：**Pydantic Schema约束LLM输出JSON，便于下游系统集成。

面试高频问题总结

基础概念类

1. RAG如何解决大模型的幻觉问题？

答：通过检索外部知识库提供真实依据，生成内容基于检索结果而非模型参数化知识，从根源减少虚构。

2. RAG vs 微调，各适用什么场景？

答：RAG适合知识更新快、需实时数据的场景（如客服、法律）；微调适合需要风格适配、任务定制的场景（如公文写作、情感对话）。

3. 为什么文本分块是RAG的基石？ chunk_overlap的意义是什么？

答：分块是连接非结构化文档与向量数据库的桥梁，确保语义单元可控。overlap防止关键信息被切断在块边界，维持上下文连贯。

技术实现类

1. 在LangChain中，如何选择Embedding模型和向量数据库？

答：

- Embedding：隐私要求高→本地部署 (bge-m3)；成本敏感→云端API；移动端→轻量模型。

- 向量库：原型→Chroma；生产→Milvus；已有PostgreSQL→PGVector。

2. 混合检索中如何平衡向量检索与关键词检索？

答：根据查询调整权重：知识密集型（如法律）偏向BM25；语义理解型（如客服）偏向向量。典型权重比为0.6:0.4，初召量设为最终需要的3-4倍。

3. 如何设计一个防幻觉的Prompt模板？

答：强制模型引用参考资料，明确指示“如果资料中没有信息，回答‘无法回答’，不要编造”，并附上来源供核实。

高级优化类

1. Agentic RAG相比传统RAG有什么本质改进？

答：传统RAG是检索驱动的条件反射；Agentic RAG是意图驱动的自主判断，具备推理规划、工具调度、反思迭代能力，能处理多步复杂任务。

2. 重排序在RAG中的作用是什么？何时可以不用？

答：重排序用Cross-Encoder精准判断文档对查询的支持度，过滤初召中的噪声。若知识库文档短且结构化、查询精确，则可省略。

3. 多轮对话中，查询重写需解决哪些核心问题？

答：消解代词（“它”→具体实体）、补全省略（“续航？”→“iPhone 15 Pro的续航表现？”）、聚焦最近提及的实体。

场景实战类

1. 企业要构建一个客服RAG系统，你会如何设计？

答：

- 文档加载：多格式解析（PDF/Word/网页），异步批处理。
- 分块：按条款/段落分割，`chunk_overlap` 15%。
- 检索：混合检索（向量+BM25），重排序（`bge-reranker-v2-m3`，阈值0.65）。
- Agent：网关模式分流简单问题，ReAct Agent处理复杂咨询。
- 安全：元数据过滤（按产品线隔离），输出脱敏，审计决策链。

关键结论：RAG的成功不在于堆砌模型，而在于对数据流（加载→分块→向量化→检索→生成）的精细化设计与调优。从传统RAG到Agentic RAG，是从“检索增强”到“智能体增强”的范式转移，也是构建下一代知识应用的核心路径。